Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ОТЧЕТ
ПО СЕМИНАРУ 9

НА ТЕМУ:

Стек, очередь и дек: представление на массиве и базовые операции

Студент ИУ5-11/4 _____ Стегунов Алексей Викторович
(группа) (подпись, дата)

Преподаватель _____ Волосова Александра Владимировна
(подпись, дата)

2026 г.

РАЗДЕЛ 1. ЦЕЛЬ И ЗАДАЧИ РАБОТЫ

Цель работы: закрепить представление стека, очереди и дека на массиве, научиться иллюстрировать состояния этих структур после последовательности операций и записывать процедуры вставки и удаления с проверкой переполнения и опустошения за время O(1).

В семинаре требуется выполнить пять заданий (10.1.1–10.1.5). Первое и третье задания проверяют понимание того, как меняются указатели top, head и tail при операциях над стеком и очередью. Второе задание посвящено совместному размещению двух стеков в одном массиве. Четвёртое и пятое задания требуют написать корректные процедуры для очереди и дека с обнаружением переполнения и опустошения.

Задачи:

1. Проиллюстрировать последовательность операций над стеком S[1..6].
2. Объяснить и записать операции двух стеков в одном массиве A[1..n].
3. Проиллюстрировать последовательность операций над очередью Q[1..6].
4. Переписать ENQUEUE и DEQUEUE с корректной проверкой underflow и overflow.
5. Записать четыре O(1)-процедуры для вставки и удаления с обоих концов дека.

РАЗДЕЛ 2. КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1. Стек

Стек — это структура данных, работающая по принципу LIFO (last in, first out): последний добавленный элемент извлекается первым. При хранении в массиве S[1..n] достаточно хранить индекс top — номер последнего элемента. Если top = 0, стек пуст. Операция PUSH увеличивает top и записывает элемент в S[top], операция POP возвращает S[top] и уменьшает top. Обе операции выполняются за O(1), потому что не требуют сдвига элементов.

После POP значение остаётся в массиве, но перестаёт принадлежать стеку: следующий PUSH может перезаписать эту клетку. Поэтому при иллюстрации состояний полезно отдельно показывать «логическое» содержимое стека и фактическое содержимое массива.

2.2. Очередь

Очередь работает по принципу FIFO (first in, first out): элементы добавляются с одного конца (tail), а извлекаются с другого (head). В циклическом массиве Q[1..n] указатель head задаёт первый элемент, а tail — следующую свободную клетку. После достижения конца массива указатель переходит в начало: tail = 1 или head = 1. Такая организация позволяет использовать освободившиеся в начале массива клетки и выполнять ENQUEUE и DEQUEUE за O(1).

Если хранить только head и tail, состояния «пусто» и «полно» могут быть неразличимы: в обоих случаях указатели могут совпадать. Поэтому в задании 10.1.4 добавляется счётчик count — количество элементов очереди. Тогда пустота определяется как count = 0, а полнота — как count = n.

2.3. Дек

Дек (deque, double-ended queue) разрешает вставку и удаление с обоих концов. Для реализации на массиве используется тот же циклический буфер, что и для очереди, но добавляются операции PUSH-FRONT и POP-BACK. Указатель head сдвигается назад при вставке в начало, tail — вперёд при вставке в конец; при удалении указатели сдвигаются в противоположных направлениях. Счётчик count позволяет отличить пустой дек от полного и проверить граничные условия за O(1).

Таблица 2.1 — Сравнение структур данных

Структура	Порядок	Вставка	Удаление	Указатели
Стек	LIFO	с одного конца	с того же конца	top
Очередь	FIFO	в tail	из head	head, tail
Дек	оба конца	в head или tail	из head или tail	head, tail, count

РАЗДЕЛ 3. ЗАДАНИЕ 10.1.1. ОПЕРАЦИИ НАД СТЕКОМ

Исходно стек S пуст, массив S[1..6], top = 0. Выполняется последовательность PUSH(S, 4), PUSH(S, 1), PUSH(S, 3), POP(S), PUSH(S, 8), POP(S). На рисунке 1 показано состояние массива после каждого шага; светло-синие клетки принадлежат стеку, серые значения после POP остаются в массиве, но уже не входят в стек.

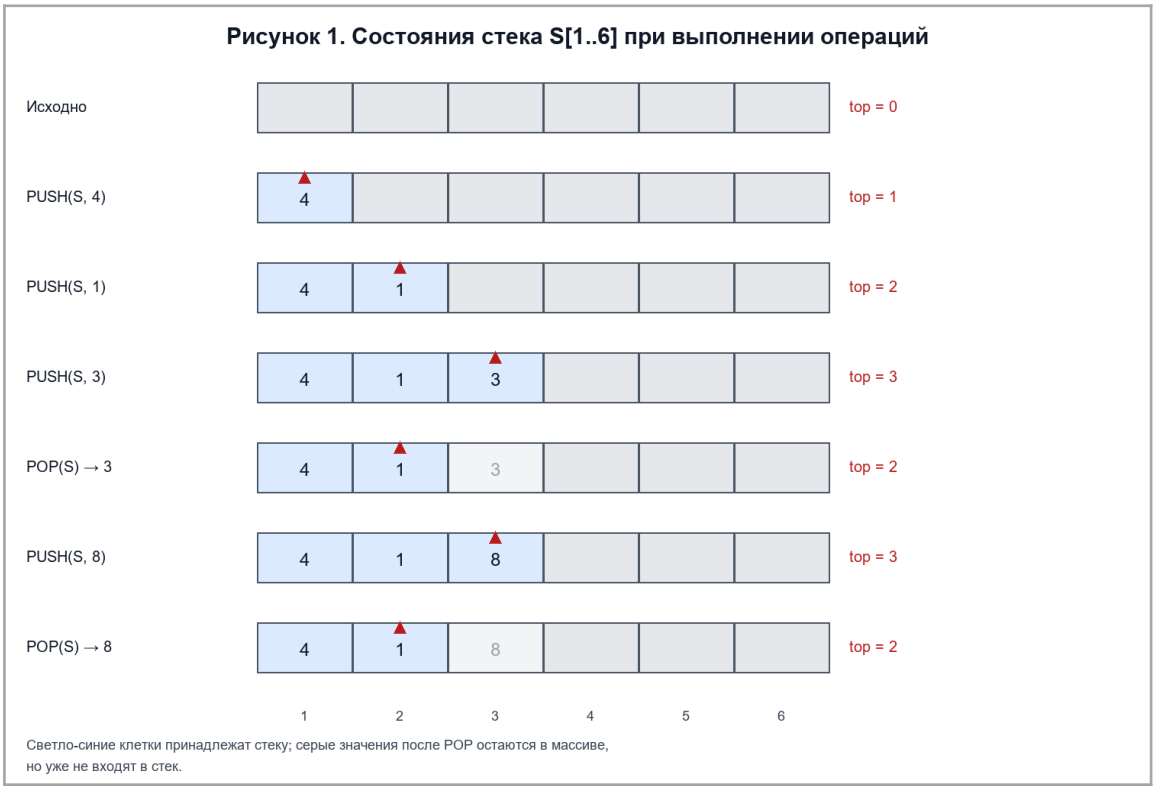


Рисунок 1 — Состояния стека S[1..6] после каждой операции

Таблица 3.1 — Последовательность операций над стеком

Шаг	Операция	Возврат	top	Содержимое стека (низ → верх)
0	начальное состояние	—	0	пусто
1	PUSH(S, 4)	—	1	4
2	PUSH(S, 1)	—	2	4, 1
3	PUSH(S, 3)	—	3	4, 1, 3
4	POP(S)	3	2	4, 1
5	PUSH(S, 8)	—	3	4, 1, 8
6	POP(S)	8	2	4, 1

После выполнения последовательности top = 2, в стеке находятся элементы 4 и 1 (4 — на дне, 1 — на вершине). В массиве остаются значения 3 и 8 в клетках 3, но они уже не принадлежат стеку. Переполнения не было: максимальное значение top равно 3 при длине массива 6.

РАЗДЕЛ 4. ЗАДАНИЕ 10.1.2. ДВА СТЕКА В ОДНОМ МАССИВЕ

Требуется разместить два стека в одном массиве A[1..n] так, чтобы ни один из них не переполнялся, пока суммарное число элементов в обоих стеках не достигнет n, а операции PUSH и POP

выполнялись за O(1). Классическое решение — растить стеки навстречу друг другу: первый стек занимает начало массива и растёт вправо, второй — конец массива и растёт влево.

Вводятся два указателя: top1 — вершина первого стека, top2 — вершина второго. Начальные значения: top1 = 0 (первый стек пуст), top2 = n + 1 (второй стек пуст). Первый стек занимает A[1..top1], второй — A[top2..n]. Свободные клетки находятся между ними: A[top1 + 1..top2 − 1]. Стеки не переполнены, пока top1 + 1 < top2. Если top1 + 1 = top2, свободных клеток нет, и любая следующая вставка приводит к переполнению.



Рисунок 2 — Два стека, растущие навстречу друг другу в массиве A[1..10]

Листинг 1 — Операции над двумя стеками в одном массиве

```
top1 = 0
top2 = n + 1

PUSH1(x)
  if top1 + 1 == top2
      error "overflow"
  top1 = top1 + 1
  A[top1] = x

POP1()
  if top1 == 0
      error "underflow"
  x = A[top1]
  top1 = top1 - 1
  return x

PUSH2(x)
  if top1 + 1 == top2
      error "overflow"
  top2 = top2 - 1
  A[top2] = x

POP2()
  if top2 == n + 1
      error "underflow"
  x = A[top2]
  top2 = top2 + 1
  return x
```

Почему это работает. Первый стек содержит top1 элементов, второй — n − top2 + 1 элементов. Суммарно занято top1 + n − top2 + 1 клеток, а свободно top2 − top1 − 1 клеток. Переполнение наступает ровно тогда, когда свободных клеток нет, то есть top2 − top1 − 1 = 0, или top1 + 1 = top2. Каждая операция изменяет только один указатель и одно значение массива, поэтому PUSH и POP выполняются за O(1). Память используется полностью: оба стека могут суммарно занять все n клеток, и фиксированного деления массива пополам не требуется.

Пример на рисунке 2: n = 10, top1 = 3, top2 = 8. Первый стек содержит 3 элемента (2, 7, 1), второй — 3 элемента (9, 5, 4), свободны клетки 4–7. Суммарно занято 6 клеток; следующая вставка в любой стек допустима, пока не будут заняты все 10 клеток.

РАЗДЕЛ 5. ЗАДАНИЕ 10.1.3. ОПЕРАЦИИ НАД ОЧЕРЕДЬЮ

Исходно очередь Q пуста, массив Q[1..6], head = 1, tail = 1, count = 0. Выполняется последовательность ENQUEUE(Q, 4), ENQUEUE(Q, 1), ENQUEUE(Q, 3), DEQUEUE(Q), ENQUEUE(Q, 8), DEQUEUE(Q). На рисунке 3 показано состояние массива после каждого шага: head указывает на первый элемент, tail — на следующую свободную клетку, серые значения после DEQUEUE остаются в массиве, но уже не принадлежат очереди.

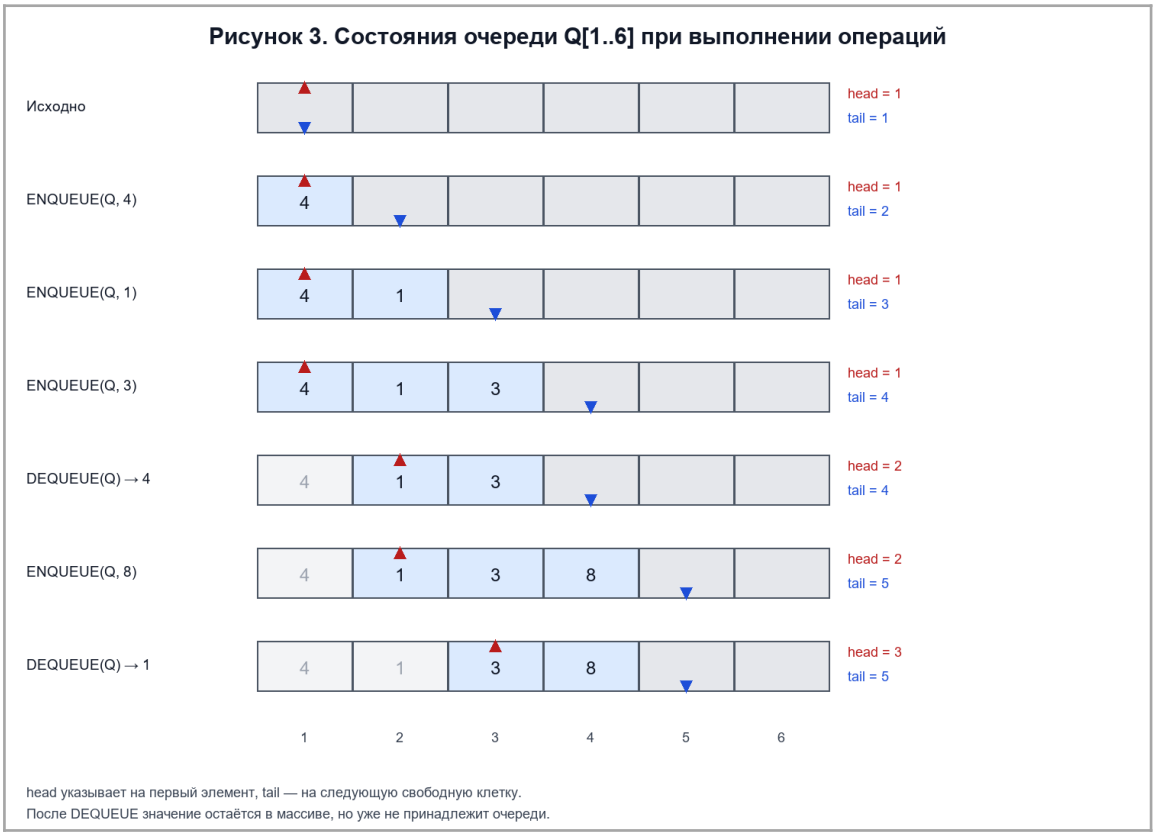


Рисунок 3 — Состояния очереди Q[1..6] после каждой операции

Таблица 5.1 — Последовательность операций над очередью

Шаг	Операция	Возврат	head	tail	count	Очередь (голова → хвост)
0	начальное состояние	—	1	1	0	пусто
1	ENQUEUE(Q, 4)	—	1	2	1	4
2	ENQUEUE(Q, 1)	—	1	3	2	4, 1
3	ENQUEUE(Q, 3)	—	1	4	3	4, 1, 3
4	DEQUEUE(Q)	4	2	4	2	1, 3
5	ENQUEUE(Q, 8)	—	2	5	3	1, 3, 8
6	DEQUEUE(Q)	1	3	5	2	3, 8

После выполнения последовательности head = 3, tail = 5, в очереди находятся элементы 3 и 8: 3 — в голове, 8 — в хвосте. В клетках 1 и 2 остались значения 4 и 1, но они уже не принадлежат очереди. В данной последовательности указатели не дошли до конца массива. Если бы tail достиг 6, следующая вставка записала бы элемент в Q[1] — так работает циклический буфер, показанный на рис. 10.2.

РАЗДЕЛ 6. ЗАДАНИЕ 10.1.4. КОРРЕКТНЫЕ ENQUEUE И DEQUEUE

В исходных процедурах из рис. 10.2 не проверяются пустота и полнота очереди. Если хранить только head и tail, эти состояния могут совпадать, поэтому нельзя отличить пустую очередь от полной. Добавим поле count — текущее количество элементов. Тогда очередь пуста при count = 0, а полна при count = Q.length. Начальные значения: head = 1, tail = 1, count = 0.

Листинг 2 — ENQUEUE и DEQUEUE с проверкой переполнения и опустошения

```
ENQUEUE(Q, x)
  if Q.count == Q.length
      error "overflow"
  Q[Q.tail] = x
  if Q.tail == Q.length
      Q.tail = 1
  else
      Q.tail = Q.tail + 1
  Q.count = Q.count + 1

DEQUEUE(Q)
  if Q.count == 0
      error "underflow"
  x = Q[Q.head]
  if Q.head == Q.length
      Q.head = 1
  else
      Q.head = Q.head + 1
  Q.count = Q.count - 1
  return x
```

Счётчик count позволяет использовать все n клеток массива. Альтернативный приём — оставлять одну клетку свободной и считать очередь полной при (tail mod n) + 1 = head; он не требует count, но уменьшает полезную ёмкость до n − 1. В данном решении выбран count, потому что он даёт точную проверку и сохраняет полную ёмкость.

Таблица 6.1 — Проверка граничных условий очереди

Ситуация	Условие	Действие
DEQUEUE из пустой очереди	count = 0	ошибка underflow
ENQUEUE в полную очередь	count = length	ошибка overflow
tail дошёл до конца массива	tail = length	tail = 1
head дошёл до конца массива	head = length	head = 1
После одного DEQUEUE	count = length − 1	ENQUEUE снова разрешён

Обе процедуры выполняют фиксированное число операций: одну проверку, одну запись или чтение, изменение указателя и изменение счётчика. Поэтому ENQUEUE и DEQUEUE работают за O(1) и не зависят от количества элементов в очереди.

РАЗДЕЛ 7. ЗАДАНИЕ 10.1.5. ЧЕТЫРЕ ОПЕРАЦИИ ДЕКА

Дек реализуется в циклическом массиве Q[1..n]. Как и для очереди, head указывает на первый элемент, tail — на следующую свободную клетку, count — количество элементов. Пустой дек: count = 0; полный: count = n. При вставке в начало указатель head сдвигается на одну клетку назад, при вставке в конец указатель tail — на одну клетку вперёд; при удалении указатели сдвигаются в противоположных направлениях. Во всех формулах используется циклический переход через границу массива.



Рисунок 4 — Пример состояния дека Q[1..6]: порядок элементов 3, 5, 7

Листинг 3 — Четыре операции дека с проверкой границ

```
PUSH-FRONT(Q, x)
  if Q.count == Q.length
      error "overflow"
  if Q.count == 0
      Q.head = 1
      Q.tail = 1
  else
      Q.head = (Q.head - 2 + Q.length) % Q.length + 1
  Q[Q.head] = x
  Q.count = Q.count + 1
  if Q.count == 1
      Q.tail = Q.head % Q.length + 1

PUSH-BACK(Q, x)
  if Q.count == Q.length
      error "overflow"
  if Q.count == 0
      Q.head = 1
      Q.tail = 1
  Q[Q.tail] = x
  Q.count = Q.count + 1
  if Q.length > 1
      Q.tail = Q.tail % Q.length + 1

POP-FRONT(Q)
  if Q.count == 0
      error "underflow"
  x = Q[Q.head]
  Q.count = Q.count - 1
  if Q.count == 0
      Q.head = 1
      Q.tail = 1
  else
      Q.head = Q.head % Q.length + 1
  return x

POP-BACK(Q)
  if Q.count == 0
      error "underflow"
  Q.tail = (Q.tail - 2 + Q.length) % Q.length + 1
  x = Q[Q.tail]
  Q.count = Q.count - 1
  if Q.count == 0
      Q.head = 1
      Q.tail = 1
  return x
```

Проверим работу на примере. Начальное состояние: head = 1, tail = 1, count = 0. Выполним PUSH-BACK(Q, 5), PUSH-FRONT(Q, 3), PUSH-BACK(Q, 7), POP-FRONT(Q), POP-BACK(Q).

Таблица 7.1 — Пример работы дека Q[1..6]

Операция	Возврат	head	tail	count	Содержимое (голова → хвост)
начальное состояние	—	1	1	0	пусто
PUSH-BACK(Q, 5)	—	1	2	1	5
PUSH-FRONT(Q, 3)	—	6	2	2	3, 5
PUSH-BACK(Q, 7)	—	6	3	3	3, 5, 7

Операция	Возврат	head	tail	count	Содержимое (голова → хвост)
POP-FRONT(Q)	3	1	3	2	5, 7
POP-BACK(Q)	7	1	2	1	5

Состояние после трёх вставок показано на рисунке 4: элементы 3, 5 и 7 занимают клетки 6, 1 и 2, head = 6, tail = 3, свободные клетки 3 и 4. После POP-FRONT возвращается 3, head становится равным 1; после POP-BACK возвращается 7, tail становится равным 2. В деке остаётся элемент 5. Все четыре операции выполняются за O(1): используются только арифметические операции над индексами, проверка count и обращение к одной клетке массива.

РАЗДЕЛ 8. ВЫВОДЫ

В ходе семинара разобраны три базовые структуры данных, реализуемые на массиве. Для стека достаточно одного указателя top; для очереди нужны head и tail, а для различения пустого и полного состояния — счётчик count или резервная свободная клетка. Дек использует тот же циклический буфер, но допускает движение указателей в обоих направлениях.

Все рассмотренные операции — PUSH, POP, ENQUEUE, DEQUEUE, PUSH-FRONT, PUSH-BACK, POP-FRONT и POP-BACK — выполняются за O(1), поскольку не перемещают элементы массива. Совместное хранение двух стеков в одном массиве позволяет использовать всю память: стеки растут навстречу друг другу, а переполнение возникает только при занятии всех n клеток.

Полученные результаты соответствуют заданиям 10.1.1–10.1.5: построены пошаговые иллюстрации для стека и очереди, объяснено размещение двух стеков, записаны корректные процедуры очереди с проверкой underflow и overflow и четыре O(1)-процедуры дека.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Методические указания к семинару 9 по дисциплине «Алгоритмизация и программирование». — М.: МГТУ им. Н.Э. Баумана, 2026.
2. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — 3-е изд. — М.: Вильямс, 2013. — Глава 10 «Элементарные структуры данных».
3. cppreference.com. Containers library. URL: https://en.cppreference.com/w/cpp/container (дата обращения: 02.11.2026).